Information Retrieval Ranking Models

Lecture 3: Sparse Ranking Methods (BM25, Language Models, Query Expansion)

Avishek Anand

Information Retrieval

February 12, 2026

Outline

1. From Vector Space to Probabilistic Ranking
2. BM25: The Probabilistic Standard
3. Statistical Language Models for Ranking
4. Query Expansion and Pseudo Relevance Feedback
5. Hands-On with PyTerrier
6. Connections to Future Lectures

Section 1

From Vector Space to Probabilistic Ranking

Recap: What We Know

Vector Space Model (Lecture 1)

- Documents and queries as vectors in term space
- Similarity measured by cosine distance
- Foundation for understanding term-based retrieval

Indexing (Lecture 2)

- Inverted index: term $\rightarrow$ posting list (docID, tf, positions)
- Accumulator-based scoring: only touch documents with term overlap
- Dense indexing: IVF+PQ for semantic vectors

Today's Question

We can store and retrieve efficiently. But **how do we score?**

The Probability Ranking Principle (PRP)

Theorem (Robertson, 1977)

If documents are ranked by decreasing probability of relevance given the query, then the expected effectiveness of the system is maximized—under the assumption of independent relevance.

Formally: Rank by $P(\text{relevant} \mid q, d)$

Implications:

- Gives a **theoretical justification** for probabilistic scoring
- All scoring functions (TF-IDF, BM25, LM) are approximations of this ideal
- Independence assumption is strong—violated in practice (documents are correlated)

Key Insight

The PRP tells us *what* to optimize. BM25 and Language Models tell us *how*.

The Ranking Problem

Formal Definition

Given:

- Query q (sequence of terms)
- Document collection D
- Inverted index (from Lecture 2)

Return: Ranked list scored by relevance $P(\text{relevant}|q, d)$

Key Insight

Not all term matches are equal:

- Rare terms ("syzygy") more informative than common ("the")
- Term frequency matters, but with diminishing returns
- Document length affects scoring

Term Weighting: TF-IDF

Term Frequency (TF)

- More occurrences $\rightarrow$ more important
- $\text{tf}(t, d) = \text{count}(t, d)$

Inverse Document Frequency (IDF)

- Rare terms $\rightarrow$ more discriminative
- $\text{idf}(t) = \log \frac{N}{\text{df}_t}$

Combined Score

$$\text{TF-IDF}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

Problem

Linear TF growth unrealistic:

- 100 occurrences $\neq$ 100$\times$ more relevant than 1
- Need saturation

TF-IDF Example

Query: "information retrieval" ($N = 100{,}000$ documents)

Term	Doc 1	Doc 2	df	IDF	Weight
information	3	1	5000	1.30	High TF
retrieval	2	0	500	2.30	High IDF
the	20	15	50000	0.30	Low IDF

Observation

- "retrieval" gets high weight (rare term)
- "the" downweighted despite frequency (common term)
- But TF still grows linearly $\rightarrow$ need saturation

Section 2

BM25: The Probabilistic Standard

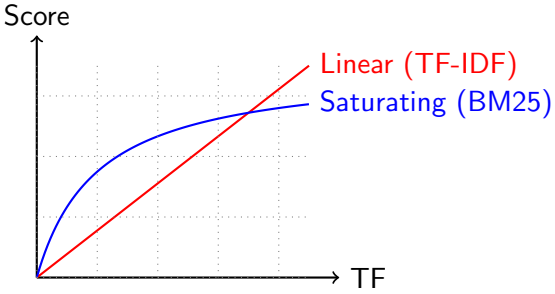
Motivation: TF Saturation

Observation:

- Relevance increases with TF
- But saturates quickly

Example:

- "algorithm" 5× vs. 50×
- 5→10 is meaningful
- 45→50 barely noticeable



Need

Non-linear TF component that saturates gracefully

BM25 Formula

Complete Formula

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

Three components:

1. **IDF:** Term importance (like TF-IDF)
2. **Saturating TF:** Non-linear term frequency
3. **Length normalization:** Penalize long documents

Standard parameters: $k_1 = 1.2$, $b = 0.75$

BM25 Component 1: IDF

IDF Formula

$$\text{IDF}(t) = \log \frac{N - \text{df}_t + 0.5}{\text{df}_t + 0.5}$$

where:

- N = total number of documents
- df_t = document frequency of term t

Properties:

- Similar to TF-IDF but probabilistically motivated
- Can be negative for very common terms ($\text{df} > N/2$)
- Rare terms get high positive IDF
- Common terms get low or negative IDF

BM25 Component 2: Saturating TF

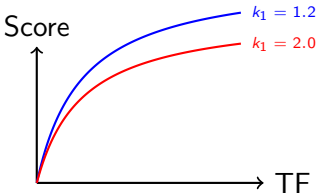
Saturating TF Formula

$$\frac{f(t,d) \cdot (k_1 + 1)}{f(t,d) + k_1}$$

Parameter k_1 (typically 1.2–2.0):

- $k_1 \to 0$: Boolean matching (TF doesn't matter)
- $k_1 \to \infty$: Linear TF (like TF-IDF)
- $k_1 = 1.2$: Moderate saturation (standard)

TF	$k_1 = 1.2$	$k_1 = 2.0$
1	0.55	0.67
5	1.53	1.67
10	1.79	1.83
50	1.98	1.96



BM25 Component 3: Length Normalization

Length Normalization Term

$$1 - b + b \cdot \frac{|d|}{\text{avgdl}}$$

Parameter b (typically 0.75):

- $b = 0$: No length normalization
- $b = 1$: Full normalization (penalize long docs heavily)
- $b = 0.75$: Partial normalization (standard)

Why needed:

- Long documents have more terms just by chance
- Should we penalize or not? Depends on collection
- b controls the trade-off

Worked Example: Full BM25 Scoring

Query: "quantum algorithm" $N = 100{,}000$, avgdl $= 500$

Two candidate documents:

- Doc A: 300 words. "quantum" appears $5\times$, "algorithm" appears $3\times$.
- Doc B: 800 words. "quantum" appears $8\times$, "algorithm" appears $1\times$.

Collection stats: df(quantum) $= 200$, df(algorithm) $= 5{,}000$

Step 1: Compute IDF.

$$\text{IDF(quantum)} = \log \frac{100000 - 200 + 0.5}{200 + 0.5} = \log \frac{99800.5}{200.5} = \log(497.8) = \mathbf{6.21}$$

$$\text{IDF(algorithm)} = \log \frac{100000 - 5000 + 0.5}{5000 + 0.5} = \log \frac{95000.5}{5000.5} = \log(19.0) = \mathbf{2.94}$$

Worked Example: Full BM25 Scoring (cont.)

Step 2: Compute scores. ($k_1 = 1.2$, $b = 0.75$, avgdl $= 500$)

Doc A ($|d| = 300$): norm $= 1 - 0.75 + 0.75 \cdot \frac{300}{500} = 0.70$

$$\text{quantum:} \quad \frac{5 \times 2.2}{5 + 1.2 \times 0.70} = \frac{11.0}{5.84} = 1.88$$

$$\text{algorithm:} \quad \frac{3 \times 2.2}{3 + 1.2 \times 0.70} = \frac{6.6}{3.84} = 1.72$$

$$\text{BM25}(q, A) = 6.21 \times 1.88 + 2.94 \times 1.72 = 11.67 + 5.06 = \mathbf{16.73}$$

Doc B ($|d| = 800$): norm $= 1 - 0.75 + 0.75 \cdot \frac{800}{500} = 1.45$

$$\text{quantum:} \quad \frac{8 \times 2.2}{8 + 1.2 \times 1.45} = \frac{17.6}{9.74} = 1.81$$

$$\text{algorithm:} \quad \frac{1 \times 2.2}{1 + 1.2 \times 1.45} = \frac{2.2}{2.74} = 0.80$$

$$\text{BM25}(q, B) = 6.21 \times 1.81 + 2.94 \times 0.80 = 11.24 + 2.35 = \mathbf{13.59}$$

Result: Doc A (16.73) > Doc B (13.59)

Doc A wins despite fewer "quantum" hits. Why? (1) Shorter doc → less length penalty. (2) More "algorithm" hits → both query terms well-covered.

BM25 Example: Effect of Document Length

Scenario: Term appears 20 times, $k_1 = 1.2$, $b = 0.75$, avgdl=1000

Short Document (500 words)

$$\begin{aligned} \text{norm} &= 1 - 0.75 + 0.75 \cdot \frac{500}{1000} \\ &= 0.625 \\ \text{TF component} &= \frac{20 \cdot 2.2}{20 + 1.2 \cdot 0.625} \\ &= \frac{44}{20.75} = 2.12 \end{aligned}$$

Long Document (2000 words)

$$\begin{aligned} \text{norm} &= 1 - 0.75 + 0.75 \cdot \frac{2000}{1000} \\ &= 1.75 \\ \text{TF component} &= \frac{20 \cdot 2.2}{20 + 1.2 \cdot 1.75} \\ &= \frac{44}{22.1} = 1.99 \end{aligned}$$

Observation

Same TF, but short document scores higher due to length penalty

BM25 Example: Rare vs. Common Terms

Collection: $N = 100{,}000$ documents

Term	df	IDF	Interpretation
quantum	100	6.21	Very discriminative
algorithm	5,000	2.99	Moderately discriminative
system	50,000	-0.41	Not discriminative
the	99,000	-6.21	Actively harmful

Key Insight

IDF difference drives discrimination:

- Rare terms dominate scoring (Matches on "quantum" worth $\sim 15\times$ matches on "system")
- Common terms contribute little or negatively

BM25F: Fielded Documents

Motivation

Real documents have **fields**: title, abstract, body, URL, anchor text.
A match in the title is worth more than a match in the body.

BM25F Formula (Simplified)

Replace raw TF with a **weighted sum across fields**:

$$\tilde{f}(t,d) = \sum_{z \in \text{fields}} w_z \cdot \frac{f(t,d_z)}{1 - b_z + b_z \cdot \frac{|d_z|}{\text{avgdl}_z}}$$

Then plug $\tilde{f}$ into standard BM25 in place of $f(t,d)$.

Parameters per field:

- w_z: field weight (title $w = 5$, body $w = 1$, URL $w = 2$)
- b_z: per-field length normalization

Used In

Elasticsearch, Solr, Bing, and most production web search engines

BM25 Parameter Tuning Guidelines

Standard Defaults

$k_1 = 1.2$, $b = 0.75$ (works well for many collections)

When to adjust k_1:

- **Verbose collections** (news, articles)
  $\rightarrow$ Higher k_1 (1.5–2.0)
- **Keyword collections** (tweets, queries)
  $\rightarrow$ Lower k_1 (0.8–1.2)

When to adjust b:

- **Variable-length docs** (web pages)
  $\rightarrow$ Higher b (0.8–0.9)
- **Uniform-length docs** (abstracts)
  $\rightarrow$ Lower b (0.5–0.7)

In Lecture 6 (Learning to Rank)

You'll learn to tune these parameters systematically using labeled data

Section 3

Statistical Language Models for Ranking

A Different Ranking Philosophy

BM25 Approach:

- "How well does document match query?"
- Scoring based on term overlap
- Weighted by importance (IDF, TF)

Language Model Approach:

- "How likely would this document generate this query?"
- Rank by $P(q|d)$
- Probabilistic generative model

Matching Score

$$\text{score}(q, d) = \sum_t w(t, q, d)$$

Generative Probability

$$\text{score}(q, d) = P(q|d)$$

Generative Story

1. Pick a document d
2. Document has a language model (probability distribution over words)
3. Generate query by sampling from this distribution
4. Documents that would more likely generate q are ranked higher

Why Language Models Matter

Principled Probabilistic Framework

- Solid theoretical foundation (compared to heuristic weighting)
- Smoothing becomes natural and essential
- Interpretable as generative process

Connection to Modern IR

- Foundation for neural language models (Lecture 4)
- BERT, GPT are sophisticated language models
- Understanding statistical LMs helps understand neural LMs

Preview

In Lecture 4, we'll see how transformers extend this probabilistic foundation with contextualized representations

Query Likelihood Model

Basic Formulation

Rank documents by probability they would generate the query:

$$P(q|d) = \prod_{w \in q} P(w|d)^{\text{count}(w,q)}$$

Maximum Likelihood Estimate:

$$P_{\text{ML}}(w|d) = \frac{\text{count}(w, d)}{|d|}$$

Critical Problem: Zero Probabilities

If query word doesn't appear in document:

- $P(w|d) = 0 \rightarrow$ Product = entire $P(q|d) = 0$
- Document ranked at bottom even if partially relevant

Solution: Smoothing is essential, not optional!

Zero Probability Example

Query: "machine learning algorithms"
Document: "This paper presents a novel neural network approach to supervised machine learning." (15 words)

Term	**count(w,d)**	$P_{\mathbf{ML}}(w\|d)$	**Result**
machine	1	$1/15 = 0.067$	✓
learning	1	$1/15 = 0.067$	✓
algorithms	0	$0/15 = 0$	✗ ZERO!
$P(q\|d)$	$0.067 \times 0.067 \times 0 = \mathbf{0}$		

Problem

Document is clearly relevant, but one missing term kills the entire score!

Smoothing: Core Idea

Principle

Mix document evidence with collection evidence:

$$P(w|d) = f(\text{document evidence}, \text{collection evidence})$$

Collection Language Model:

$$P(w|C) = \frac{\sum_{d' \in C} \text{count}(w, d')}{\sum_{d' \in C} |d'|}$$

Two Main Approaches:

1. **Jelinek-Mercer:** Fixed linear interpolation
2. **Dirichlet:** Length-adaptive interpolation

Effect

Jelinek-Mercer Smoothing

Formula

$$P(w|d) = (1 - \lambda) \cdot P_{\text{ML}}(w|d) + \lambda \cdot P(w|C)$$

Parameter λ (typically 0.1–0.3):

- $\lambda = 0$: Only use document (no smoothing)
- $\lambda = 1$: Only use collection (ignore document)
- $\lambda = 0.1$: Mostly trust document, small collection backup

Properties:

- ✓ Simple and interpretable
- ✓ Same λ for all documents
- ✗ Doesn't adapt to document length

Dirichlet Smoothing

Formula

$$P(w|d) = \frac{\text{count}(w, d) + \mu \cdot P(w|C)}{|d| + \mu}$$

Parameter μ (typically 1000–2500):

- Smoothing strength (number of "pseudo-counts" from collection)
- Higher μ: More smoothing. Lower μ: Less smoothing.

Key Difference: Length-Adaptive

- **Short documents:** μ dominates denominator $\rightarrow$ rely more on collection
- **Long documents:** Actual counts dominate $\rightarrow$ rely on document
- Automatically adapts smoothing to document length!

Dirichlet Smoothing: Length Adaptation Example

Term "algorithm" with $P(w|C) = 0.001$**,** $\mu = 2000$

Document	**count(w,d)**	$\|d\|$	$P(w\|d)$
Short doc	2	100	$\frac{2+2}{2100} = 0.0019$
Long doc	2	2000	$\frac{2+2}{4000} = 0.001$

Observation

- Same term count (2), but short doc $\rightarrow$ higher probability
- Long doc $\rightarrow$ more smoothing, closer to collection probability
- This is **automatic** based on $|d|$

Jelinek-Mercer vs. Dirichlet

Aspect	Jelinek-Mercer	Dirichlet
Formula	$(1-\lambda)P_{\text{ML}}(w\|d) + \lambda P(w\|C)$	$\frac{\text{count}(w,d)+\mu P(w\|C)}{\|d\|+\mu}$
Parameter	λ (0.1–0.3)	μ (1000–2500)
Length adaptation	Fixed for all docs	Automatic
Smoothing amount	Same for all docs	More for short docs
Interpretation	Linear interpolation	Pseudo-counts
Best for	Uniform-length collections	Variable-length collections

Which to Use?

Dirichlet generally better for most collections. Standard in Indri, Galago.

Language Models vs. BM25

Aspect	BM25	Language Models
Philosophy	Term matching score	Generative probability
TF handling	Explicit saturation (k_1)	Implicit via smoothing
Length norm	Explicit (b)	Implicit via smoothing
IDF	Explicit component	Through $P(w\|C)$
Zero probabilities	Avoided via IDF	Smoothing essential
Parameters	k_1, b	λ or μ
Foundation	Probabilistic relevance	Language modeling

Connection: RM3 Uses Language Modeling

Recall RM3 Formula

$$P(w|R) \propto \sum_{d \in R} P(w|d) \cdot P(d|q)$$

Key Insight:

- RM3 uses language modeling framework
- $P(w|d)$ estimated for each pseudo-relevant doc
- RM3 builds a *language model of relevance*

Why This Matters

Understanding language models helps you understand query expansion:

- Both use probabilistic term distributions
- Both require smoothing
- Natural progression: Statistical LM $\rightarrow$ RM3 $\rightarrow$ Neural LM (Lecture 4)

Section 4

Query Expansion and Pseudo Relevance Feedback

The Vocabulary Mismatch Problem

Core Issue

Queries and relevant documents use different vocabulary:

Medical Domain

- Query: "heart attack"
- Document: "myocardial infarction"

Technical Domain

- Query: "ML"
- Document: "machine learning"

General Domain

- Query: "elderly care"
- Document: "geriatric nursing"

Result

- BM25 score = 0
- Relevant doc not retrieved

Sparse Retrieval Limitation

Exact term matching only — no semantic understanding

Relevance Feedback Overview

Explicit Relevance Feedback

User marks documents as relevant/non-relevant

- ✗ Impractical: users won't give feedback
- ✗ Chicken-and-egg: need good initial results

Pseudo Relevance Feedback (PRF)

Assumption: Top-k retrieved documents are relevant

- ✓ No user interaction needed
- ✓ Automatic query expansion
- ✗ Risk: If assumption fails $\rightarrow$ query drift

Process:

1. Retrieve initial results with original query
2. Extract terms from top-k documents
3. Expand query with these terms
4. Re-retrieve with expanded query

RM3: Relevance Model 3

Foundation

Probabilistic query expansion using language modeling framework

Two-stage process:

1. **Estimate Relevance Model**

$$P(w|R) \propto \sum_{d \in R} P(w|d) \cdot P(d|q)$$

   - For each word w, estimate probability from "relevance"
   - Weight by how relevant each doc appears: $P(d|q)$ from initial retrieval
   - Select top `fb_terms` words by $P(w|R)$

2. **Interpolate with Original Query**

$$P(w|Q_{\text{new}}) = (1 - \lambda) \cdot P(w|Q_{\text{original}}) + \lambda \cdot P(w|R)$$

RM3: Key Parameters

fb_docs (Feedback Documents): 3–10

How many top documents to use for expansion

- Too few: May miss relevant terms. Too many: Risk non-relevant docs.
- **Standard:** 10

fb_terms (Feedback Terms): 10–20

How many expansion terms to add

- Too few: Limited benefit. Too many: Noise.
- **Standard:** 10

λ (Interpolation Weight): 0.5–0.8

Original vs. expansion weight

- Lower (0.5): Trust original query more
- Higher (0.8): Trust expansion more

RM3 Example

Original Query: "machine learning"

Top-3 Retrieved Documents (fb_docs=3):

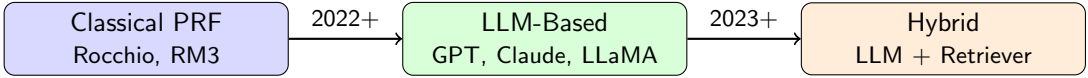
1. "Neural networks and deep learning algorithms..."
2. "Supervised machine learning techniques..."
3. "Machine learning applications in AI..."

Extracted Terms (fb_terms=5):

- neural (P=0.15), deep (P=0.12), supervised (P=0.10), algorithms (P=0.09), AI (P=0.08)

Expanded Query ($\lambda = 0.6$):
"machine learning neural deep supervised algorithms AI"

Beyond PRF: Modern Query Expansion Strategies



Key Insight

LLMs have **parametric knowledge** — they can generate expansion terms without retrieving any documents first. No risk of PRF's "bad top-k" problem!

Strategy 1: LLM Synonym & Paraphrase Generation

Idea

Ask an LLM to generate synonyms, related terms, or paraphrases of the query.

Prompt:

```
Given the search query:  "heart attack symptoms"
Generate 5 alternative phrasings and medical synonyms that a relevant
document might use.
Return only the terms, one per line.
```

LLM Output:

- myocardial infarction signs
- cardiac arrest warning indicators
- chest pain emergency symptoms
- coronary event presentation
- acute heart failure signs

Expanded BM25 query: "heart attack symptoms myocardial infarction cardiac coronary chest pain"

Advantage

Bridges the exact vocabulary gap that BM25 cannot handle on its own

Strategy 2: Hypothetical Document Generation (HyDE)

Core Idea (Gao et al., 2023)

Ask the LLM to **generate a hypothetical relevant document**, then use its embedding for dense retrieval or extract its terms for sparse expansion.

Prompt:

```
Write a short paragraph that would be a perfect answer to the question:
"What causes ocean acidification?"
```

LLM Output:

> *"Ocean acidification is primarily caused by the absorption of atmospheric carbon dioxide (CO_2) by seawater. When CO_2 dissolves, it forms carbonic acid, which lowers the pH of the ocean. This process threatens marine organisms with calcium carbonate shells, such as corals and mollusks..."*

For Sparse: Extract key terms → "carbonic acid", "calcium carbonate", "pH", "seawater", "corals"

For Dense: Embed the hypothetical doc → use as query vector (Lecture 5)

Strategy 3: Chain-of-Thought Query Decomposition

Idea

Complex queries benefit from being broken into sub-queries that each retrieve different evidence.

Prompt:

```
I need to answer:  "How did the 2008 financial crisis affect European
housing markets?"
Break this into 3-4 simpler search queries that together would cover the
full answer.
```

LLM Output:

1. "2008 financial crisis causes subprime mortgage"
2. "European housing prices 2008 2009 decline"
3. "EU bank bailouts housing market impact"
4. "European mortgage lending regulations post-crisis"

Pipeline

Run BM25 on each sub-query $\rightarrow$ merge results $\rightarrow$ better recall than single complex query

Strategy 4: Query2Doc (Wang et al., 2023)

Idea

Generate a pseudo-document with an LLM, **prepend it to the original query**, and run BM25 on the concatenation.

Original query: "dark matter detection methods"

LLM-generated pseudo-doc:

> *"Dark matter detection experiments include direct detection using cryogenic detectors, indirect detection through gamma-ray observations, and collider searches at the Large Hadron Collider..."*

Expanded query to BM25: original query + pseudo-doc text

✓ **Advantages:**

- Drops into any BM25 system
- No retrieval needed first
- LLM runs once per query

✗ **Risks:**

- LLM hallucinations add noise
- Longer "query" = slower
- Need to weight original vs. generated text

Comparison: Classical vs. LLM Query Expansion

Aspect	Classical (RM3)	LLM-Based
Knowledge source	Top-k retrieved docs	Parametric (training data)
Failure mode	Query drift (bad top-k)	Hallucination
Latency	Fast (stats only)	Slower (LLM inference)
Cost	Free	API cost / GPU
Domain knowledge	Needs in-domain corpus	Pre-trained, zero-shot
Cold-start	Fails (empty index)	Works (no index needed)
Interpretability	Terms from real docs	Generated terms

Best Practice (2024+)

Combine both: Use LLM expansion to improve the initial retrieval, then apply RM3 on the (now better) top-k results. The two methods are complementary.

When Query Expansion Helps vs. Hurts

✓ Helps When:

- Initial results mostly relevant
- Vocabulary mismatch present
- Broad informational queries
- Query: "climate change"
- Expansion adds: "warming", "CO2", "emissions"

✗ Hurts When:

- Initial results off-topic
- Ambiguous queries
- Very specific queries
- Query: "jaguar"
- Expansion adds: "car", "speed" (wrong sense!)

Query Drift

Query Drift Example

Query: "python programming"
Top result: Article about Python snake species
RM3 expansion: "reptile", "snake", "habitat", "venom"
LLM expansion: "programming language", "syntax", "libraries" ✓ Correct!

Section 5

Hands-On with PyTerrier

PyTerrier Setup

```python
import pyterrier as pt
if not pt.started():
    pt.init()

# Load dataset (VASWANI: small, fast for teaching)
dataset = pt.get_dataset('irds:vaswani')

# Index (or use pre-built)
if not os.path.exists("./vaswani_index"):
    indexer = pt.IterDictIndexer("./vaswani_index")
    index_ref = indexer.index(dataset.get_corpus_iter())
else:
    index_ref = pt.IndexRef.of("./vaswani_index")

index = pt.IndexFactory.of(index_ref)

# Get topics and qrels
topics = dataset.get_topics().head(10)
```

Exercise 1: BM25 Baseline

```python
# Default BM25 (k1=1.2, b=0.75)
bm25 = pt.BatchRetrieve(index, wmodel="BM25")

# Retrieve results
results = bm25.transform(topics)

# Examine one query
print(results[results['qid']=='1'][
    ['qid', 'docno', 'score', 'rank']].head())
```

Output:

qid	docno	score	rank
1	51	8.234	1
1	486	7.921	2
1	12	7.456	3

Exercise 1: Parameter Effects

```
# Create BM25 variants
bm25_default = pt.BatchRetrieve(index, wmodel="BM25")
bm25_high_k1 = pt.BatchRetrieve(index, wmodel="BM25",
                                controls={"bm25.k1": "2.0"})
bm25_low_b = pt.BatchRetrieve(index, wmodel="BM25",
                              controls={"bm25.b": "0.3"})
bm25_high_b = pt.BatchRetrieve(index, wmodel="BM25",
                               controls={"bm25.b": "0.95"})

# Compare
comparison = pt.Experiment(
    [bm25_default, bm25_high_k1, bm25_low_b, bm25_high_b],
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "P_10"],
    names=["Default", "k1=2.0", "b=0.3", "b=0.95"]
)
print(comparison)
```

Exercise 2: RM3 Query Expansion

```python
# RM3 pipeline: retrieve -> expand -> retrieve again
bm25 = pt.BatchRetrieve(index, wmodel="BM25")
rm3 = pt.rewrite.RM3(index)  # Default: fb_docs=10, fb_terms=10
pipeline_rm3 = bm25 >> rm3 >> bm25

# Compare with baseline
comparison = pt.Experiment(
    [bm25, pipeline_rm3],
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "recall_100"],
    names=["BM25", "BM25+RM3"]
)
print(comparison)

# Inspect expanded query
expanded = (bm25 >> rm3).transform(topics)
print("Original:", topics[topics['qid']=='1']['query'].values[0])
print("Expanded:", expanded[expanded['qid']=='1']['query'].values[0])
```

Exercise 2b: LLM-Based Query Expansion

```python
from openai import OpenAI
client = OpenAI()

def llm_expand(query, n_terms=5):
    """Expand query using LLM synonym generation."""
    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content":
            f"Generate {n_terms} search terms related to: "
            f"'{query}'. Return only the terms, comma-separated."}]
    )
    expansion = resp.choices[0].message.content
    return f"{query} {expansion}"

# Expand all topics
topics_expanded = topics.copy()
topics_expanded['query'] = topics_expanded['query'].apply(llm_expand)

# Run BM25 on expanded queries
results_llm = bm25.transform(topics_expanded)

# Compare: BM25 vs RM3 vs LLM-expand vs LLM+RM3
pipeline_llm_rm3 = bm25 >> rm3 >> bm25  # on LLM-expanded topics
```

Discussion: When does LLM expansion beat RM3? When does combining help?

Exercise 3: Language Models

```
# Dirichlet LM with different mu values
lm_1000 = pt.BatchRetrieve(index, wmodel="DirichletLM",
                           controls={"c": "1000"})
lm_2500 = pt.BatchRetrieve(index, wmodel="DirichletLM",
                           controls={"c": "2500"})
lm_5000 = pt.BatchRetrieve(index, wmodel="DirichletLM",
                           controls={"c": "5000"})

# Compare with BM25
comparison = pt.Experiment(
    [bm25, lm_1000, lm_2500, lm_5000],
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "P_10"],
    names=["BM25", "LM mu=1000", "LM mu=2500", "LM mu=5000"]
)
print(comparison)
```

Exercise 4: Comprehensive Comparison

```python
# Build all systems
systems = {
    "BM25": bm25,
    "Dirichlet LM": lm_2500,
    "BM25 + RM3": bm25 >> pt.rewrite.RM3(index) >> bm25,
    "LM + RM3": lm_2500 >> pt.rewrite.RM3(index) >> lm_2500,
}

# Final evaluation
final_results = pt.Experiment(
    list(systems.values()),
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "P_10", "recall_100"],
    names=list(systems.keys()))
)
print(final_results)
```

Discussion: Which baseline wins? Does RM3 help more for BM25 or LM?

What We Covered Today

Sparse, Term-Based Ranking

- **BM25:** Probabilistic relevance, saturating TF, length normalization
- **Language Models:** Generative probabilities, smoothing essential
- **Query Expansion:** RM3 for vocabulary mismatch + LLM-based alternatives

Fundamental Limitations

- Still bag-of-words (order doesn't matter)
- Exact term matching only (even with expansion)
- No semantic understanding
- "car" and "automobile" completely different
- "bank" (river) vs. "bank" (financial) treated identically